

MODUL 6 · TYPESCRIPT DALAM NEXT.JS

Mengapa TypeScript Menjadi Standar Wajib



Ini perbedaan antara bug yang kamu temukan dan bug yang ditemukan user.

Kita Sudah Memakai TypeScript

Di bab-bab sebelumnya TypeScript sudah ada, tapi kita belum membahasnya secara khusus.

Yang sudah kita lihat

File berekstensi .ts dan .tsx (page.tsx, layout.tsx) dari create-next-app, file tsconfig.json, dan garis merah di editor saat ada kesalahan. Kita memakainya, tapi belum benar-benar memahaminya.

Yang akan kita lakukan di bab ini

Memahami apa yang sebenarnya dikerjakan TypeScript, kenapa ia dianggap wajib, lalu masuk lebih dalam:

- tsconfig.json
- tipe dasar
- props
- alias tipe
- tipe PageProps & searchParams.

Apa itu TypeScript?

JavaScript yang ditambah sistem *tipe,* sistem yang mengecek sebelum kode dijalankan.

1. **Static type checker**
   Mengecek tipe data (string, number, dst.) saat kita menulis kode, bukan saat aplikasi berjalan

2. **Tipe dihapus saat compile**
   Hasil akhirnya tetap JavaScript biasa — browser tidak tahu soal TypeScript

3. **Gratis di runtime**
   Tidak membuat aplikasi lebih lambat; manfaatnya ada di editor dan saat build

Mendefinisikan Tipe dengan Titik Dua ":"

Pola dasarnya: nama diikuti titik dua, lalu tipenya.

Pola

namaVariabel: tipe → namaParameter: tipe → fungsi(): tipeKembalian

1. **Tipe dasar: string, number, boolean**
   Ditulis dengan huruf kecil
2. **Bisa dipasang di variabel, parameter, dan return value fungsi**
3. **Nanti kita bahas lebih dalam: di mana saja tipe bisa dipakai (objek, props, dll.)**

JavaScript Tanpa Tipe: Apa Risikonya?

JavaScript tidak peduli tipe. Kesalahan baru ketahuan saat kode dijalankan, atau tidak ketahuan sama sekali.

Typo di atribut objek

Tanpa definisi isi objek

- Salah ketik user.**nmae** tidak menghasilkan error
- Hasilnya **diam-diam** undefined
- Bug baru muncul jauh di bagian lain aplikasi

Salah tipe di parameter

Tanpa definisi tipe parameter

- Fungsi bisa dipanggil dengan tipe apa saja
- "5" + 3 menghasilkan "53", bukan 8
- **Lupa** mengisi argumen → hasil NaN

Contoh: Bug Diam-diam di JavaScript

Tidak ada peringatan di editor. Semua baris ini "valid" bagi JavaScript.

contoh.js

```
const user = { name: "Rizki", age: 25 };

console.log(user.nmae);                 // undefined (typo, tanpa error)
console.log(user.nmae.toUpperCase()); // crash saat dijalankan

function tambah(a, b) { return a + b; }

console.log(tambah(5, 3));    // 8
console.log(tambah("5", 3)); // "53" — bukan 8!
console.log(tambah(5));       // NaN
```

Contoh: Versi TypeScript

Kesalahan yang sama kini ditangkap di editor, sebelum kode dijalankan.

contoh.ts

```
const user = { name: "Rizki", age: 25 };
user.nmae;
// Error: Property 'nmae' does not exist on type
// '{ name: string; age: number; }'.

function tambah(a: number, b: number): number {
  return a + b;
}
tambah("5", 3);
// Error: Argument of type 'string' is not assignable
// to parameter of type 'number'.
tambah(5);
// Error: Expected 2 arguments, but got 1.
```

Intinya: Error Pindah ke Awal

TypeScript meminimalkan typo dan kesalahan tipe pada variabel, fungsi, dan objek.

JavaScript

*Ketahuan saat **berjalan***

- Error muncul di browser atau server
- Bisa lolos sampai ke pengguna
- Mencari penyebab = membaca ulang banyak kode

TypeScript

*Ketahuan saat **menulis** kode*

- Garis merah langsung di editor
- Pesan error menunjuk baris yang salah
- Perbaiki sebelum aplikasi dijalankan

Kenapa Jadi Standar di Project Modern?

1. ***Autocomplete*** dan ***refactor*** lebih aman
   Editor tahu atribut apa saja yang ada pada sebuah objek

2. Tipe yang error akan **menggagalkan** next build
   Bug tipe tertahan sebelum sampai ke production

3. Dokumentasi lebih **hidup**
   Tipe menjelaskan bentuk data tanpa harus membaca seluruh implementasi

TypeScript juga punya batas

Tipe hanya ada saat development

Setelah di-compile, tipe dihapus, yang artinya **respons API, input form, isi database** tidak otomatis dicek TypeScript saat aplikasi berjalan.

Karena itu kita juga pakai Zod

Validasi data saat runtime tetap diperlukan. TypeScript menjaga kode kita konsisten, sedangkan Zod menjaga data yang masuk. Keduanya saling melengkapi.